Quantum Computing in Supply Chain Management

QCSC Update

August 5th, 2026

Arizona State University

Today's presenter

Dr. Dale Rogers

ON Semiconductor Professor of Business and Professor,
NASPO Department of Supply Chain Management

Today's presenter

Aarav Shandilya

Student Researcher, SCENE Program

High School student with passion in Quantum Computing and Mathematics

Today’s presenter

Chetan Pulimi

Graduate Student @ W.P. Carey School of Business

Master of Science Business Analytics ‘26

Today's agenda

- Supply Chain Introduction
- Problem Introduction
- Classical Optimization Approach
- Quantum Optimization Approach
    - Quadratic Optimization
    - Higher-Order Optimization
- Closing Remarks

Supply Chain & Problem Introduction

An introduction to supply chain management systems by Dr. Dale Rogers

Project 1: Dell Excess & Obsolete Inventory

The Challenge: Sub-optimal disposal/scrapping of excess parts leads to ~$10M quarterly loss out of an $80M inventory pool. Current tools struggle with long horizons (up to 4-5 years), weekly alerts vs. quarterly actions, and multi-site strategic mapping (e.g., US-Japan).

Proposed Quantum Solution

Proactive Re-balancing

Identify optimal candidates for transfer: which parts, from where, to where - maximizing recovery value.

Excess Report

Origin-destination pairs with lead times and full end-to-end costs for actionable decisions.

Quantum Advantage

Explore quantum methods to handle long planning horizons and complex multi-site constraints faster.

Key Constraints & Context

Planning horizon up to 4-5 years • Weekly alerts / quarterly actions • Site-pair mapping (US-Japan) • Current "what-if" runs take ~2.5 hrs (improved from 3-4 days) • Limited capacity for further classical enhancements • High-school collaborator via SCENE program

Project 2: Gurobi & QUBO Supply Chain Models

Student Teams

Two teams from the ASU MS Business Analytics program

- Building a classical Gurobi optimization model
- Developing a corresponding QUBO model
- New supply chain problem from Dell Technologies

Focus: demonstrate efficacy of the QUBO approach through rigorous experiments.

Timeline & Deliverables

Spring 2026 → Summer 2026 → Fall 2026

- New models delivered
- Experimental validation of QUBO performance
- Funding proposal in progress
- Journal paper under development

Collaboration Rhythm

Weekly meetings every Friday afternoon, with additional sessions as needed. Ongoing close partnership with Dell Technologies and Tyto Athene team members to ensure industrial relevance and rapid iteration on both classical and quantum formulations.

Project at a Glance

01 Excess & Obsolete

Industry Partner
Dell Technologies

Focus Area
Inventory rebalancing & recovery

Impact Opportunity
~$10M quarterly loss reduction

Approach
Quantum-enhanced proactive model + Excess Report

Talent
High-school student (SCENE program)

02 Gurobi + QUBO Models

Industry Partner
Dell Technologies & Tyto Athene

Focus Area
New supply-chain optimization problem

Impact Opportunity
Benchmark classical vs. quantum formulations

Approach
Gurobi model + QUBO model + experiments

Talent
Two MS Business Analytics student teams + High School Student

Original Focus

Project Focus

Comparing
Quantum Inspired Algorithms with Classical Optimization & Heuristic algorithms

Focusing
Solution Quality & Computational Effort

Evaluation Metrics

Solution Quality
Objective Function Value

Computational Effort
Runtime & Memory Consumption

Dell Problem Introduction

Forward Stocking Locations for same-business-day service parts

Dell operates ~600 Forward Stocking Locations and hubs for same-business-day service parts.

Every zip maps to a hub within a 130-mile SLA and assignment is at the SKU level, so one zip can get part A and part B from different hubs.

Goal: cut cost by calculated risk -closing hubs, restocking, relaxing the radius for selected zip-part pairs.

Three decisions, one model: which hubs stay open

· what's stocked where

· which hub serves each zip-part pair.

Five cost families: inventory · storage (fixed + overflow) · transfer · transport (flat ≤ 20 mi, per-mile after) · SLA-risk past 130 mi - priced, not forbidden.

Digital twin mirroring the Dell network.

$$\min \sum_{jk} Y_{jk} P_k + \sum_{j} X_j S^{lim} + \sum_{j} W_j S^{var} + \sum_{jk} Y_{jk} (1 - T_j) C$$

$$+ \lambda_1 \sum_{ijk} h^s b_{ik} Z_{ijk} + \lambda_2 \sum_{ijk} h^d b_{ik} (d_{ij} - 20)^+ Z_{ijk} + \lambda_3 \sum_{ijk} b_{ik} (d_{ij} - 130)^+ Z_{ijk}$$

$$\text{s.t.} \quad \sum_j Z_{ijk} = 1 \;\; \forall i, k \qquad Z_{ijk} \le Y_{jk} \qquad Y_{jk} \le X_j \qquad W_j \ge \sum_k Y_{jk} - L, \;\; W_j \ge 0$$

inventory · fixed storage · overflow · transfer | transport: flat past 20 mi · per-mile · SLA-risk past 130 mi

Classical Optimization

How Heuristic Algorithms were used for this Project

Classical Heuristics

Gurobi Optimization

Gurobi Introduction

- **Mathematical Optimization Solver**
- Solves LP, **MILP**, QP, MIQP,
- Employs **advanced techniques**
- Is widely used in **Supply Chain Optimization**

Gurobi Formulation

How we used Gurobi

The model - native MILP, solved exactly.

Variables: Z assign zip-part → hub

· Y stock part at hub

· X hub open

· W overflow.

Constraints: every zip-part pair → exactly one hub

· serve only stocked parts

· only open hubs stock

· overflow counted past storage limit L.

The ground truth - branch-and-bound with optimality proofs, not a heuristic pass.

Both solvers consume identical instances: same demand pairs, same candidate sets, same cost scalars.

Tiers: 200 / 500 / 800 hubs; the low tier alone is 115,710 zip-part pairs → 610K variables, 725K constraints.

Quantum Computing

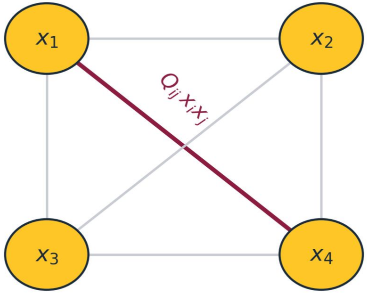
And how we framed Optimization Problems on QUBOs and HUBOs

Quadratic Unconstrained Binary Optimization

QUBO in Supply Chain

QUBOs

Quadratic Unconstrained Binary Optimization is a **mathematical** optimization **framework** that represents decision **variables** as **binary** values (0 or 1) and **encodes** both the **optimization** objective and **problem** constraints into a **single energy function**



binary variables · pairwise interactions

Problem Representation

Every decision is a binary variable; interactions are at most pairwise - exactly the granularity of our network decisions: assign, stock, open.

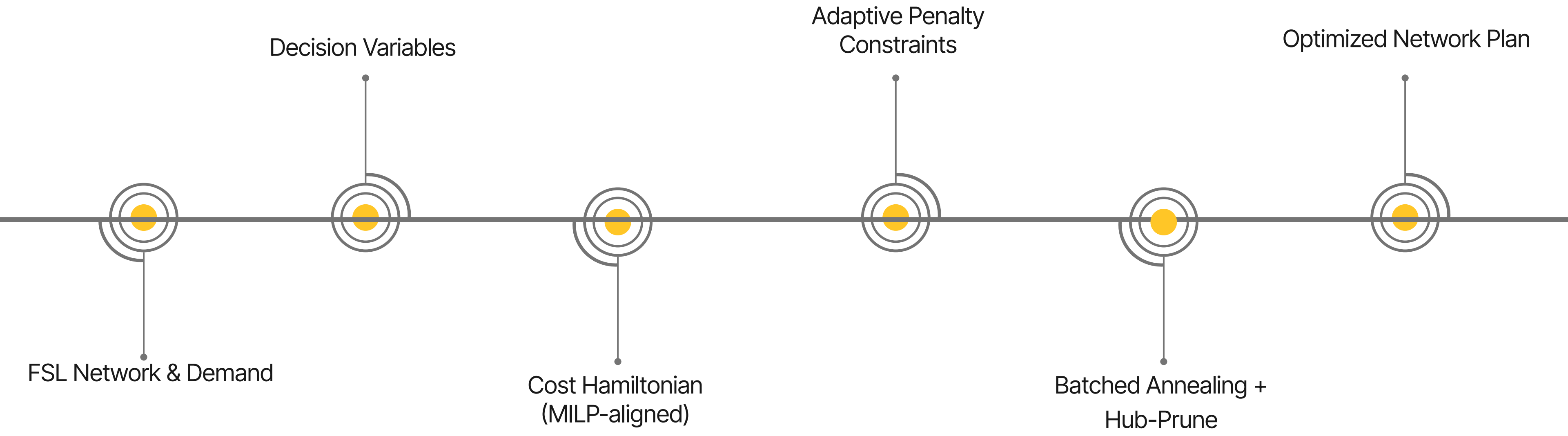
Unified Objective Function

Constraints become quadratic penalties added to the objective - e.g. one hub per pair: $P_c \cdot (\Sigma_j z - 1)^2$. Feasible states sit in the low-energy valleys.

Supply Chain Applications

The most widely used encoding in quantum-inspired supply-chain optimization - siting, stocking, and assignment reduce naturally to binary choices.

QUBO Implementation Pipeline



Objective Function Description

$$H(z,y,x) = \sum_{ijk} c_{ijk} z_{ijk} + \sum_{jk} [P_k + (1 - T_j) C] y_{jk} + \sum_{j} S^{lim} x_j$$

$$+ \boxed{P_{c1}} \sum_{ik} \left(\sum_{j} z_{ijk} - 1 \right)^2 + \boxed{P_{c2}} \sum_{ijk} z_{ijk} (1 - y_{jk}) + \boxed{P_{c3}} \sum_{jk} y_{jk} (1 - x_j)$$

where $c_{ijk} = \lambda_1 h^s b_{ik} + \lambda_2 h^d b_{ik} (d_{ij} - 20)^+ + \lambda_3 b_{ik} (d_{ij} - 130)^+$

cost half = the MILP objective, term for term | constraint half = quadratic penalties, each Pc ≈ 502K × 5 ≈ 2.5M

- Top line - the cost half: transport, stocking + transfer, fixed hub cost
- the MILP objective, term for term, so cost comparisons are apples-to-apples.
- Bottom line - the constraint half: one-hot, stock-link, hub activation as quadratic penalties, each ≈ 502K × 5 ≈ 2.5M. Overflow handled classically downstream.

Key Novelties

Adaptive Penalty Constraints

- QUBO has no hard constraints -> rules become penalties
- Too low → infeasible. Too high → cost signal drowns
- Check which families were violated, grow only those ×1.5
- Re-solve, cap 5 passes, keep most feasible sample
- No per-instance hand-tuning; cost model stays undistorted

Batching

- A single QUBO over the whole network is far larger than the sampler can search effectively
- Split into sub-QUBOs, each a complete miniature
- Parts accumulate until the Z-variable cap trips → 11 / 2 / 4 batches
- Independent, so they run as a parallel SLURM array
- Trade-off: no batch sees a hub's global demand

Runtime Results

~15 hr

QUBO pipeline wall time on the high tier data(800 hubs)

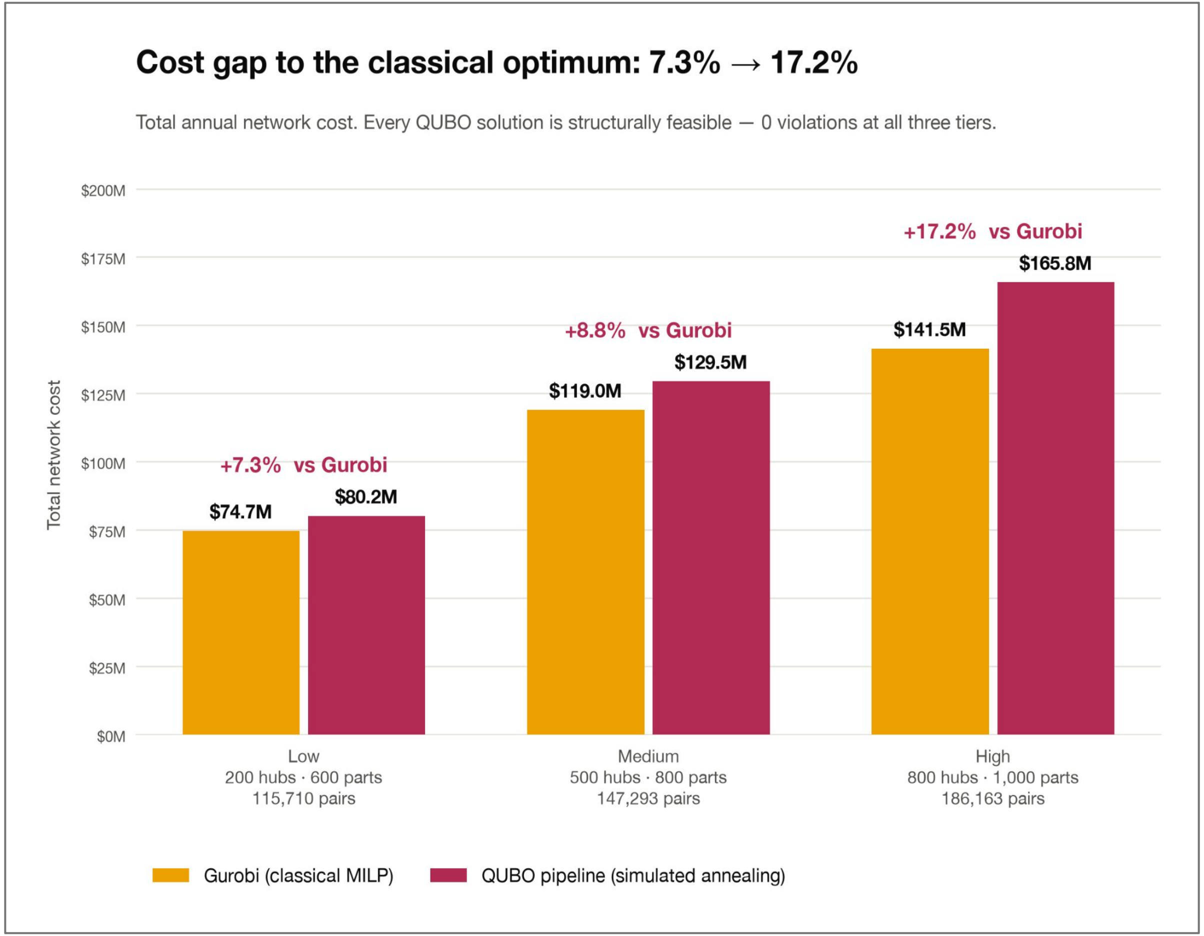
Pipeline stays roughly flat from medium (15.8 hr → ~15 hr) at 80–90 GB peak memory

~30 hr

Gurobi wall time on the same tier - stalls at ≈ 0.23% gap

At low & medium tiers Gurobi is far faster: 74 s · 16 min - the case is behavior at scale

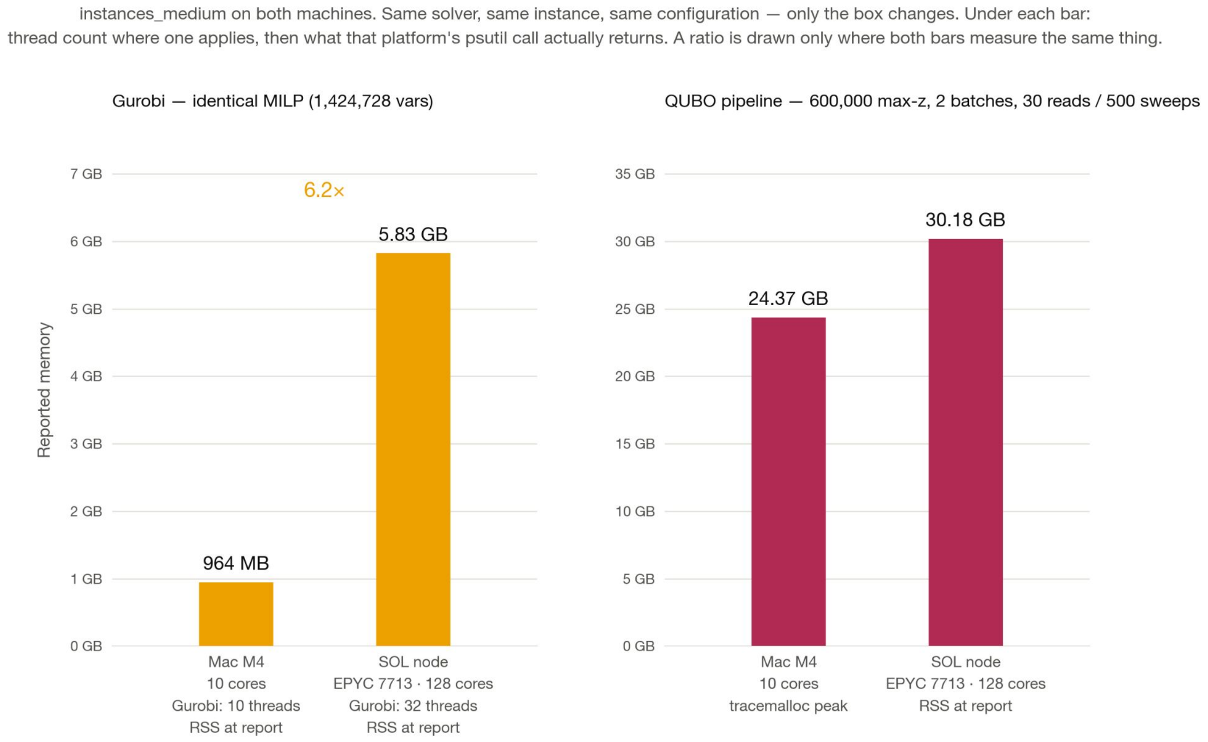
Objective Function Results



The QUBO pipeline returns structurally feasible solutions at all three tiers - zero structural, SLA, or max-distance violations.

Cost is the tradeoff: it trails Gurobi by 7.3% at Low, 8.8% at Medium, and 17.2% at High, so the gap roughly doubles as the instance grows from 116K to 186K hub-zip-part pairs.

Memory Comparison



Same solver, same instance, same configuration on both machines - only the hardware changes. Gurobi's footprint moves 6.2x, from 964 MB on 10 cores to 5.83 GB on 128, because it scales with thread count rather than model size.

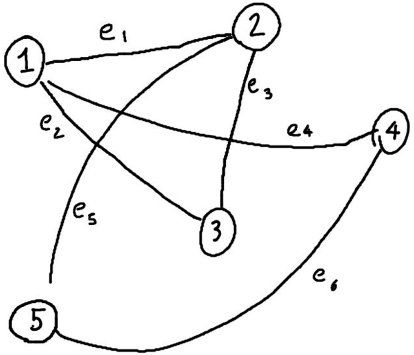
The QUBO bars are not a ratio: Mac reports tracemalloc peak, SOL reports RSS. Only the SOL figure is comparable to Gurobi, and there the pipeline holds 30.18 GB against 5.83 GB.

Higher Order Unconstrained Binary Optimization

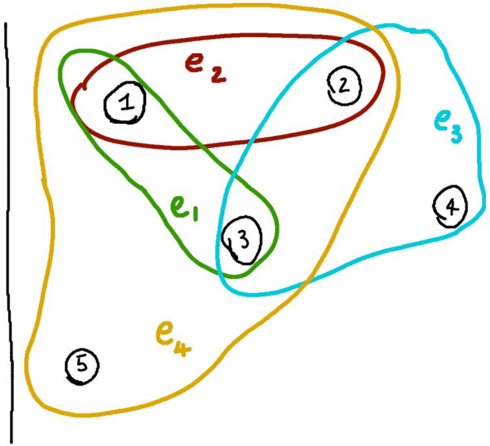
HUBO in Supply Chain

HUBOs

Higher-Order Unconstrained Binary Optimization is a **mathematical** optimization **framework** that represents decision **variables** as **binary** values (0 or 1) and **encodes** both the **optimization** objective and **problem** constraints into a **single energy function**.



Graph



Hypergraph

Natural Problem Representation

Higher-order interactions can be modeled directly without decomposing them into multiple pairwise terms, reducing the need for auxiliary variables and simplifying the mathematical formulation.

Unified Objective Function

The optimization objective and system constraints are combined into a single Hamiltonian (energy function). Minimizing this energy yields solutions that satisfy the constraints while optimizing the desired objective.

Supply Chain Applications

HUBOs naturally model multi-way dependencies in logistics, such as inventory allocation, transportation routing, and demand fulfillment, making them well suited for complex supply chain optimization problems.

HUBO Implementation

How to map a real supply chain problem onto a HUBO

Decision Variables

Each shipment between an excess and demand site is represented by binary variables encoding shipment quantities.

Variables determine:

- Which routes are selected
- How much inventory is shipped
- Which supply-demand pairs are active

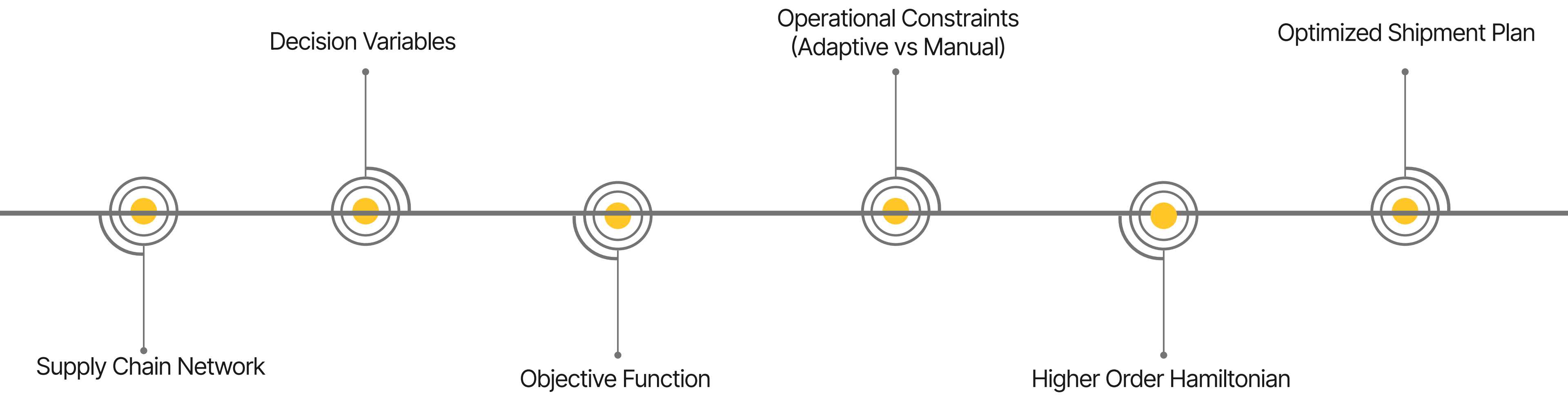
Objective function

- **Minimizes** transportation costs and unmet demand.
- **Reduces** violations of operational constraints through penalty terms.
- **Maximizes** inventory utilization and shipment efficiency.
- **Determines** the most cost-effective and feasible shipment plan.

Constraints Encoding

HUBOs naturally model multi-way dependencies in logistics, such as inventory allocation, transportation routing, and demand fulfillment, making them well suited for complex supply chain optimization problems.

HUBO Implementation Pipeline



Methodology

How to actually compare HUBO effectiveness?

Implementation

*The HUBO formulation was executed on the **Sol supercomputer** using Simulated Quantum Annealing (**SQA**) to **optimize** supply chain decisions.*

- Executed on the **Sol Supercomputer** using the **NEC VE20 Vector Engine**.
- Solved using **Simulated Quantum Annealing (SQA)** to identify the minimum-energy solution.
- Results were benchmarked against **Gurobi** using runtime and objective value comparisons.

Benchmarking

- **Gurobi** is a ***Heuristics algorithm*** which **solves** to **optimality**
- **Gurobi** is a **MILP** (*Mixed Integer Linear Programming*) **optimizer,** capable of only **Linear** or **Pairwise interactions**
- Gurobi is **unable** to **optimize HUBOs** due to the **higher** order **factor**
- Therefore, the higher order terms need to be solved in a different way

Solution

***Quadratization* Converts** Higher-Order Terms into **Pairwise Interactions**

- Transforms higher-order interactions into linear or pairwise (quadratic) terms using auxiliary variables.
- Produces a QUBO formulation that can be efficiently optimized using solvers such as Gurobi.

Runtime Results

8.34%

Average runtime difference between SQA and Gurobi

On the 50 datasets, SQA on average was 8.34% faster than Gurobi

14.57%

Greatest runtime difference between SQA and Gurobi

Greatest runtime difference between SQA and Gurobi

Objective Function Results

7.82%

Average difference from optimality for SQA

On the 50 Datasets, the average percentage difference for each dataset from optimality for SQA was 7.82%

2 Datasets

Number of Datasets that SQA Solved to Optimality

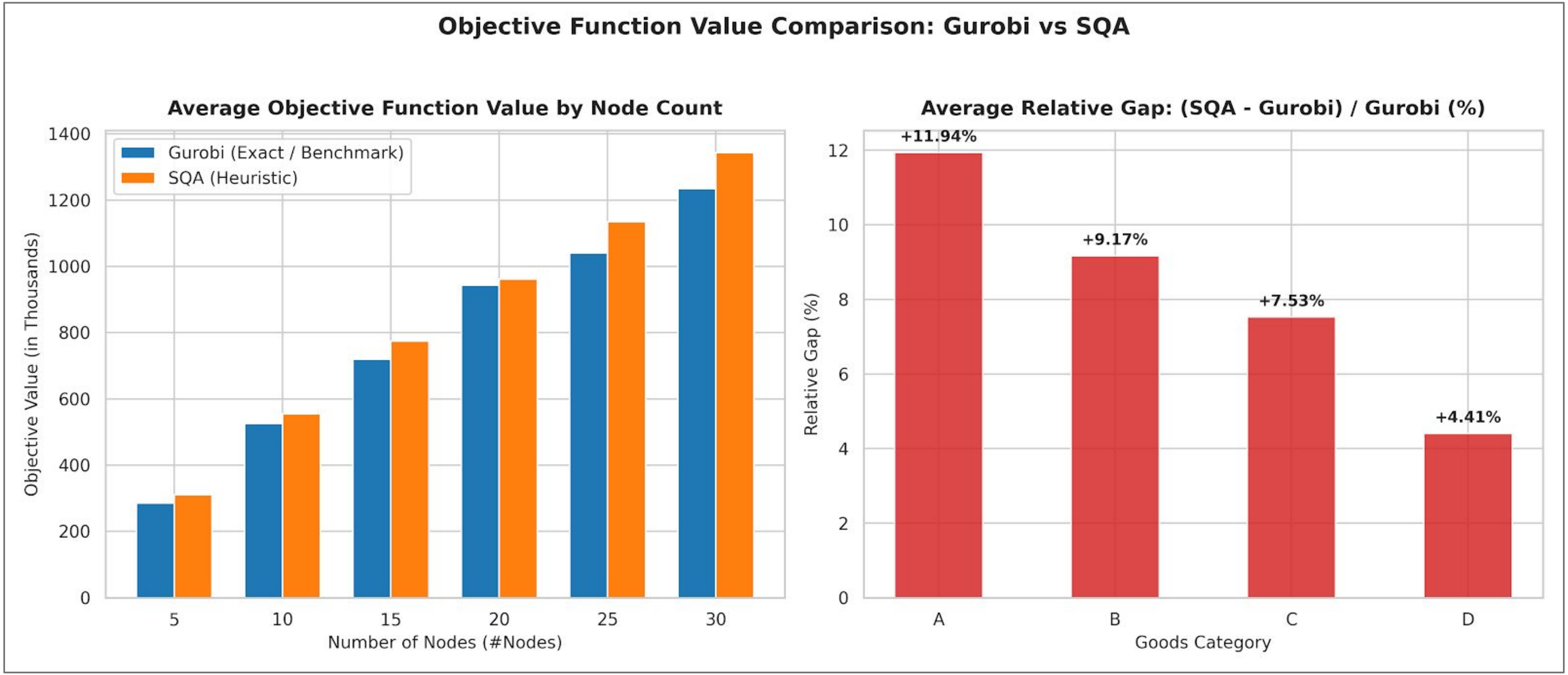
SQA method solved to optimality 2 times
Note: Gurobi, a Heuristic Solver, solves to optimality in all Cases

Charts & Graphs

Runtime Performance Comparison

Charts & Graphs

Objective Function Visualization



W. P. Carey School of Business

Arizona State University

Thank you

Questions or discussion

Contact information:

- Aarav Shandilya: ashandi5@asu.edu
- Chetan Pulimi Reddy: cpulimi@asu.edu

ASU® Arizona State University